

Determination Report: Self-Supporting Runtime Development

Publisher: Keddeh Systems | Rights holder: Keddeh Systems (project attribution; legal ownership not independently verified)

Author: OpenAI Codex, technical assessment and document preparation | Architecture and supplied source: Aboudy Keddeh

Classification: Project research | Licence: Reserved rights; no new licence grant

Version: 1.0 | Date: 8 October 2026 (Australia/Adelaide)

Status: Prepared for owner review and publication; no independent approval or certification

Imported source baseline, followed by identified local fixes: Keddeh1/KEDDEH--, commit 29a22dcfdd89d8c7cf90c83d1bda7d7f9a5e37b9

1. Abstract and determination

The source provides a concrete basis for a supervised multi-runtime system: a React workstation, Node service, Python mining/control components, native C shared-memory programs, executable WASM, and an MCP qualification surface. Local qualification establishes working engineering invariants and selected WASM operations. It does not yet establish integrated, autonomous production operation. The highest-return programme is to make the system reproducible, addressable, fenced, observable and recoverable before expanding its transport or computation surface.

2. Scope and method

The assessment uses the imported 273-file baseline, direct reading of the principal execution paths, and six locally reproduced commands. The ten targets below are implementation specifications and release gates, not assertions of completed delivery. Two source files were corrected following the owner instruction to fix identified errors. Other application source was preserved. Test-generated evidence is retained separately. Scheduling and budget estimates are intentionally omitted because staffing, fleet topology and deployment requirements are unspecified.

3. Reproduced evidence

E01: 15 engineering tests passed. E02: MCP self-test returned PASS. E03: memory contract returned contiguous=true and ok=true, with 124-byte packets, 726 complete packet slots and 88 remainder bytes. E04: native Makefile now exits 0 with -Werror and layout assertions enabled. Initial build defects were corrected: guarded feature macro, derived gap/tail dimensions, explicit MMIO-offset assertion, and corrected telemetry format. The before-fix log is retained. Layout execution confirms packet=64, mmio_offset=53248 and canvas=131072. E05: seven declared estate/ABI cases passed, including 1000 norm/triangle iterations; this is a targeted ABI suite. E06: three norm examples passed. These results concern different boundaries and must retain separate statuses.

4. Development target specifications

T01: Reproducible qualification

Source: package.json; package-lock.json; .github/workflows/engineering-layer-qualification.yml.

Implementation: The existing CI exercises engineering and MCP paths without building the workstation or native runtime. A prior frozen install in the separate copy rejected manifest/lock mismatches. Establish an exact dependency lock, supported Node/Python/C toolchains and three independent build jobs. Never suppress native layout assertions to obtain a green build.

Release gate: Clean checkout: dependency install, UI build/typecheck, native compilation and existing tests succeed; each artifact has a source revision and digest.

Dependencies: T02.

T02: Authoritative component inventory

Source: mcp/runtime-manifest.json; workers/SECTOR_001.json through SECTOR_005.json; src/data/InfrastructureNodeMetadata.ts.

Implementation: Generate an inventory from repository manifests, sector files and executable entrypoints. Retain sector identity, physical process identity, KEX coordinate, source digest, health interface and dependency edges separately. Declare authority and reject ambiguous bindings; identical filenames in duplicate directories are not interchangeable by assumption.

Release gate: A missing executable, conflicting binding and unknown sector each produce a structured rejection; all five sector configurations remain individually addressable.

Dependencies: T01.

T03: Durable lease and generation authority

Source: runtime/engineering/distributed-coordinate-directory.mjs; separate working-copy runtime/session-control/.

Implementation: Integrate the separately developed PostgreSQL control plane through an authenticated adapter. Lock the session row before comparing database time and ownership. Increment generation within the recovery transaction. Enforce the token at checkpoint publication and worker dispatch, rather than only in an HTTP handler.

Release gate: Two real database clients contend for an expired lease; exactly one receives ownership. Stale tokens cannot write or dispatch. Repeat after database and worker restart.

Dependencies: T01,T02.

T04: Sector lifecycle supervisor

Source: runtime/engineering/tot-node-supervisor.mjs; mining-engine/main.py; mining-engine/stratum_engine.py.

Implementation: Connect supervisor state to actual process readiness and dependency checks. Use bounded exponential backoff, retry budgets and circuit breaking. Separate process alive, TLS ready, pool authorized and useful work accepted. A sector fault must not provoke uncontrolled restart of healthy sectors.

Release gate: Kill one worker, invalidate its credential and block its dependency separately; observe bounded recovery or explicit escalation with unaffected peer sectors.

Dependencies: T02,T03.

T05: Capability-based carrier resolver

Source: server.ts RealMulticastEngine; mining-engine/carrier_t12_bridge.py; src/services/VfsMulticastHypervisor.ts.

Implementation: Bind stable logical addresses to explicit runtime capability probes. Distinguish host-local shared memory and Unix sockets, backend UDP/TCP, and browser-supported transport. Record selection, failover reason and endpoint generation. Do not infer inter-host reachability from a successful local socket bind.

Release gate: Run a carrier capability matrix; unsupported carriers are rejected, supported transitions preserve logical identity and require renewed physical readiness.

Dependencies: T02,T04.

T06: Consistent checkpoint and restoration

Source: mining-engine/durable_lineage_ledger_core.py; mesh_persistence.json; src/services/VfsStorageIndexedDb.ts.

Implementation: Keep ledger events, VFS persistence and Chromium profile recovery as separate checkpoint domains. Publish immutable, quiesced snapshots with schema version, checksum, runtime descriptor and owner scope. Test restoration, not merely snapshot creation. In-memory JavaScript continuity is outside the basic profile contract.

Release gate: Corrupt, truncate and version-mismatch snapshots; restoration fails safely. A valid checkpoint restores the declared durable state and reports measured RPO/RT0.

Dependencies: T03,T04.

T07: Executable remote workstation

Source: src/assets/workstations/Chrome.html; src/data/html5ChromeBrowserTemplate.ts; src/components/Html5AppRunner.tsx.

Implementation: Add isolated Chromium workers, authenticated signaling, viewport delivery and generation-scoped input. Freeze input during suspicion and require a new frame, focus reconciliation and authoritative generation before resumption. Do not replay uncertain transactional clicks. Preserve third-party embedding policies.

Release gate: A representative navigation/type/click test passes; worker failure causes visible recovery and a new transport, without replay of uncertain input. Record latency distributions.

Dependencies: T03,T05,T06.

T08: Cross-language ABI and synchronization

Source: src/services/wasm/brainkKernel.wasm; src/services/runtimeConcurrentEngine.ts; opt/keddeh/src/keddeh_mesh_runtime.c.

Implementation: Derive packet sizes, offsets, endian encoding and capacity from one versioned contract. The Python memory contract declares 139264 bytes; the native canvas declares 131072 bytes and has been corrected to pass its size and MMIO-offset assertions. Establish whether these are distinct domains or reconcile them explicitly. Define atomic producer/consumer publication, ownership and bounds.

Release gate: Native and WASM fixtures agree on their shared fields; malformed lengths, stale generation, ring exhaustion and concurrent access are tested. Native assertions remain enabled.

Dependencies: T01,T02.

T09: Evidence-driven health and bounded repair

Source: src/services/AutomatedHealthDaemon.ts; runtime/engineering/tot-safety-kernel.mjs; server.ts telemetry routes.

Implementation: Separate observations, configured expectations and synthesized values. Correlate measurements by sector, generation and timestamp. Health checks should not substitute a default hashrate for an absent measurement. Restrict repair to named actions with preconditions, budgets and audit records.

Release gate: Absent telemetry remains unknown; stale readings expire. Every repair records trigger, authorization, command result, postcondition and stopping reason.

Dependencies: T02,T04,T05.

T10: Release qualification and rollback

Source: mining-engine/firmware_manager.py; src/services/firmwareUpdateEngine.ts; evidence/; .github/workflows/.

Implementation: Create immutable release manifests linking hashes, compatible ABI, required runtime and passed gates. Verify signatures using managed trust keys. Deploy to one sector first, observe readiness and work, then promote. Preserve checkpoints and the previous qualified artifact for rollback.

Release gate: An intentionally bad release is rejected or rolled back; rollback restores compatibility and durable state. Key rotation and tampered artifacts have explicit rejection tests.

Dependencies: T01,T06,T08,T09.

5. Sequencing and execution control

Begin with T01 and T02 together. Resolve ABI/build failures in T08 before treating native artifacts as deployable. Deliver T03, T04 and T09 as the first operational control loop. Then implement T05 and T06, followed by T07. Qualify T10 after rollback prerequisites exist. T01 and T02 have a mutual feedback dependency: an initial inventory supports qualification, while qualified artifacts populate the authoritative inventory; neither requires waiting for the other to be complete.

6. Decision rules

Promotion requires all mandatory checks for the selected deployment target to pass. An unavailable external service is an unexecuted external gate, not a local pass. The corrected native compilation permits further native validation; it does not replace concurrency, lifecycle or production operation checks. Repair must never change its own release gates or downgrade verification to manufacture success.

7. Limitations and unresolved decisions

Required deployment decisions are database authority, worker identity model, Chromium isolation, checkpoint storage, network topology and release signing authority. The referenced formal publishing standard and DOCX template could not be retrieved by the skill reader. This release uses the accessible skill's metadata and rights requirements; exact template conformance remains unverified. Publication approval is not inferred from document generation.

8. Sources and research disposition

Source references: paths above are relative to the stated GitHub baseline. Evidence E01-E06 is supplied in evidence/ with commands, exit codes and SHA-256 receipts. E05-generated-receipt.json is the current generated receipt; original repository evidence was restored after the test. The separate session-control module is outside the source baseline and is not represented as deployed. Source inspection establishes structure, not operating performance. No third-party literature review, standards certification, live pool share acceptance or deployed recovery experiment is claimed.